PINN-SR+ : A PINNs Inspired Method for Discovery of PDEs

Erik M. Urzua* and Eric Mjolsness†
AI in Science Institute, University of California, Irvine

March 19, 2026

We have created a data-driven method that simultaneously trains a Physics Informed Neural Network [13] (PINNs) and determines a best fitting partial differential equation (PDE) from a single noisy data set for the discovery of PDEs when one is not known within some scientific area. This is done by repeatedly switching back and forth between removing functions from a library of functions and training the PINNs model on the current best fitting equation for the data until some stopping condition is met. The idea of alternating between sparsely reducing a current best-fitting equation and updating the weights and biases of a neural network was first done by Chen, Liu and Sun in [8] through an algorithm they called ADO which can be regarded as a simplified version of the alternating direction methods of multipliers [6]. Since our methods builds on of their PINN-SR model [8], we call our method PINN-SR+

We consider parameterized PDEs in the general form

$$\mathcal{D}_t^n\left[\mathbf{u}\right] + \mathcal{F}\left[\mathbf{u}, \mathbf{u}^2, \ldots, \nabla_{\mathbf{x}}\mathbf{u}, \nabla_{\mathbf{x}}^2\mathbf{u}, \nabla_{\mathbf{x}}\mathbf{u}\cdot\mathbf{u}, \ldots; \lambda\right] = \mathbf{p} \tag{1}$$

where $\mathbf{u} = \mathbf{u}(\mathbf{x}, t) \in \mathbb{R}^{1\times m}$ is the latent solution, $\mathcal{D}_t^n\left[\mathbf{u}\right]$ is the n-th order time derivative, $\mathcal{F}\left[\cdot\right]$ is the nonlinear functional of $\mathbf{u}$ and its spatial derivatives parameterized by λ and $\mathbf{p}\left(\mathbf{x}, t\right)$ is the source term which might be identically 0 and, as is in [8] and the various SINDy methods [7, 15, 14, 11], assume that the PDE that governs only consists of a few terms that can be selected from a large-space of candidate functions through sparse regression. With this assumption, Eq. (1) can be reformulated as

$$\mathcal{D}_t^n\left[\mathbf{u}\right] = \boldsymbol{\phi}\boldsymbol{\Lambda} \tag{2}$$

where $\boldsymbol{\phi} = \boldsymbol{\phi}\left(\mathbf{u}\right) \in \mathbb{R}^{1\times s}$ is a large library of symbolic candidate functions that possibly includes $\mathbf{p}$,

$$\boldsymbol{\phi} = \left[\mathbf{1}, \mathbf{u}, \mathbf{u}_x, \mathbf{u}_y, \cdots, \sin(\mathbf{u}), \cdots\right] \tag{3}$$

and $\boldsymbol{\Lambda} \in \mathcal{R}^{s\times m}$ is a sparse coefficient matrix.

Given a dataset of spatiotemporal measurements $\mathcal{D}_u$, we train a neural network $\mathbf{u}_\theta$ on $\mathcal{D}_u = \{(\mathbf{x}_i, t_i\}_{i=1}^{N_u}$ and then use $\mathbf{u}_\theta$ to evaluate $\boldsymbol{\phi}$ over a large collection of collocation points $\mathcal{D}_c$ randomly sampled in the spatiotemporal space

*urzuae@uci.edu
†emj@uci.edu

and use sparse regression to find $\mathbf{\Lambda}$. Within this setup, $\mathbf{u}_\theta$ largely functions as a nonlinear function to approximate to the latent solution $\mathbf{u}$ whose various partials can be determined over $\mathcal{D}_c = \{(\mathbf{x}_i, t_i\}_{i=1}^{N_c}$ quickly at machine precision using automatic differentiation [4]. After an initial pretraining step, we alternate between training $\mathbf{u}_\theta$ using the loss function

$$\mathcal{L}\left(\theta, \mathbf{\Lambda} : \mathcal{D}_u, \mathcal{D}_c\right) = \mathcal{L}_D\left(\theta, \mathcal{D}_u\right) + \alpha \mathcal{L}_{EQ}\left(\theta, \mathbf{\Lambda}; \mathcal{D}_c\right) \tag{4}$$

where $\mathcal{L}_D\left(\theta, \mathcal{D}_u\right) = \frac{1}{N_u}\|\mathbf{u}_\theta\left(\mathcal{D}_u\right) - \mathbf{u}\left(\mathcal{D}_u\right)\|_2^2$ is the data loss and $\mathcal{L}_{EQ}\left(\theta, \mathbf{\Lambda}; \mathcal{D}_c\right) = \frac{1}{N_c}\|\mathcal{D}_t^n\left[\mathbf{u}_\theta\right]\left(\mathcal{D}_c\right) - \boldsymbol{\phi}\left(\mathcal{D}_c\right)\mathbf{\Lambda}\|_2^2$ is the PDE loss based on the current values of the trainable network parameters θ and coefficients in Λ and determining a more sparse $\mathbf{\Lambda}$ coefficient matrix using a modified version of the recursive feature elimination (RFE) sparse regression method from [17]. This alternating between optimizing $\mathbf{u}_\theta$ and finding sparser $\mathbf{\Lambda}$ is done for a fixed number of iterations or until our modified version of RFE does not return a sparser $\mathbf{\Lambda}$ from the previous one. Once the alternating optimization steps has been completed, a final greedy sparse regression method based on optimizing the least squares loss is done to Λ for a final set of possible modeling PDEs for the data. With these final modeling PDEs and the ones determined at each previous sparsification step, we select the best based on its complexity and $\mathcal{L}_{EQ}\left(\theta, \mathbf{\Lambda}; \mathcal{D}_c\right)$ loss as the one that maximizes the following metric:

$$s\left(\mathbf{\Lambda}_i\right) = \frac{-\log\left(\frac{l_{i-1}}{l_i}\right)}{c_{i-1} - c_i} \quad \text{for } i = 1, 2, \ldots \tag{5}$$

where l_i is the equation loss for the i-th iterations, $\mathcal{L}_{EQ}\left(\theta, \mathbf{\Lambda}_i; \mathcal{D}_c\right)$ and c_i is the complexity of the equations that result from the coefficients found in $\mathbf{\Lambda}_i$. This *score* metric is the same used by the PySR symbolic regression library [9]. Once the $\mathbf{\Lambda}^* = \arg\max_{i=1,2\ldots} s\left(\mathbf{\Lambda}_i\right)$ has been determined, we then do a final post training step that simultaneously optimizes $\mathbf{u}_\theta$ and and the coefficients found in $\mathbf{\Lambda}^*$ to give a PDE expression that best models the data.

We have tested or PINNS-SR+ PDE discovery method on the following collection of PDEs under different percentages of additive Gaussian noise and candidate libraries $\boldsymbol{\phi}$: the Allen-Cahn equation [1] as a reaction-diffusion example, Burgers equation [3] as an example that develops a shock in finite time, the diffusion (heat) equation, the Korteweg-De Vries (KdV) equation [10] as an example with soliton solutions, and the Klein-Gordon equation [12] as a second order in time wave equation example. Additionally we have compared our PDE discovery method against other methods that similarly train a neural network to approximate the latent solution $\mathbf{u}$ and use sparse regression on a library of candidate function under the same percentages of additive Gaussian noise and candidate libraries $\boldsymbol{\phi}$: Deep-Mod[5], PDE-READ[17] and PDE-LEARN[16]. The percentage of times that each equation discovery method was able to correctly identify the true PDE form is shown in Table 1 with the row labeled as PINN-SR+ the results from our method using default hyperparameter values α and Opt. PINN-SR+ the results using optimal hyperparameter values for each PDE.

To demonstrate the capabilities of our PINN-SR+ method to be able to discover modeling PDEs for more complex phenomena, we have also tested our method on discovery the kinetic Vlasov equation from synthetic data taken

Method	Allen-Cahn	Burgers	Heat	KdV	Klein-Gordon
PINN-SR+	17.5%	55%	92.5%	97.5%	92.5%
Opt. PINN-SR+	**100%**	**82.5%**	**95%**	**100%**	**100%**
Deep-Mod	97.5%	30%	47.5%	60%	5%
PDE-READ	**100%**	25%	70%	77.5%	92.5%
PDE-LEARN	0%	30%	85%	0%	0%

Table 1: The percentage that each method was able to learn the correct equation form. Each method was run on the same but different 40 random partitions of the noisy PDE in to a training and testing dataset. PINN-SR+ using the optimal hyperparameter values is highly performant across the test suite. Second best in most cases is PINN-SR+ using a single hyperparameter value.

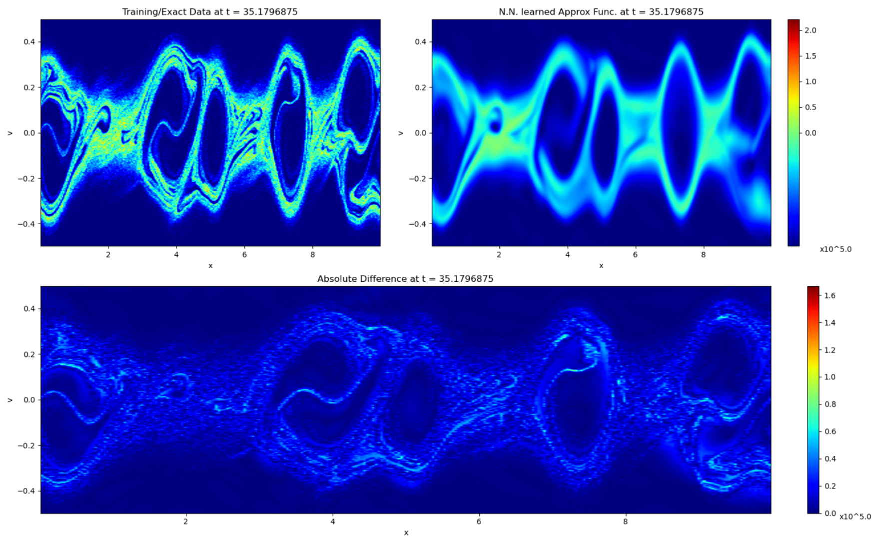

Figure 1: Comparison between the neural network $\mathbf{u}_\theta$and the latent solution $\mathbf{u}$ from one of the five different runs we did on this Vlasov example.

from a Particle in Cell (PIC) simulation similar to the way Alves and Fiuza did in [2]. On each of the five different partitions of the simulation data into a training and testing dataset, our method was able to discover the correct form of the Vlasov equation in one phase space even though the coefficients were substantially smaller than the true equation's coefficient values. The best learned equation in terms of minimal average coefficient error was

$$\partial_t u = -0.71053696 v \partial_x u + 0.6956345 E \partial_v u$$

A figure displaying at one point in time of the $\mathbf{u}_\theta$ approximation of the latent solution $\mathbf{u}$ can be seen in the Figure 1.

Going forward we will continue to test our method on harder and more complex equations and data from across the sciences as well as see how dependent the method is on the architecture of the neural network and optimization methods. In our tests, we did not directly impose/enforce the boundary conditions from which each of the PDE datasets satisfy. So we will also see how much the boundary conditions affect the discovered equation as we have been able to determine all our test equations without enforcing them.

References

[1] Samuel M. Allen and John W. Cahn. "A microscopic theory for antiphase boundary motion and its application to antiphase domain coarsening". In: *Acta Metallurgica* 27.6 (1979), pp. 1085–1095. ISSN: 0001-6160. DOI: https://doi.org/10.1016/0001-6160(79)90196-2. URL: https://www.sciencedirect.com/science/article/pii/0001616079901962.

[2] E. P. Alves and F. Fiuza. "Data-driven discovery of reduced plasma physics models from fully kinetic simulations". In: *Phys. Rev. Res.* 4 (3 2022), p. 033192. DOI: 10.1103/PhysRevResearch.4.033192. URL: https://link.aps.org/doi/10.1103/PhysRevResearch.4.033192.

[3] C Basdevant et al. "Spectral and finite difference solutions of the Burgers equation". In: *Computers Fluids* 14.1 (1986), pp. 23–41. ISSN: 0045-7930. DOI: https://doi.org/10.1016/0045-7930(86)90036-8. URL: https://www.sciencedirect.com/science/article/pii/0045793086900368.

[4] Atılım Günes Baydin et al. "Automatic differentiation in machine learning: a survey". In: *J. Mach. Learn. Res.* 18.1 (Jan. 2017), pp. 5595–5637. ISSN: 1532-4435.

[5] Gert-Jan Both et al. "DeepMoD: Deep learning for model discovery in noisy data". In: *Journal of Computational Physics* 428 (2021), p. 109985.

[6] Stephen Boyd et al. "Distributed optimization and statistical learning via the alternating direction method of multipliers". In: *Foundations and Trends® in Machine learning* 3.1 (2011), pp. 1–122.

[7] Steven L Brunton, Joshua L Proctor, and J Nathan Kutz. "Discovering governing equations from data by sparse identification of nonlinear dynamical systems". In: *Proceedings of the national academy of sciences* 113.15 (2016), pp. 3932–3937.

[8] Zhao Chen, Yang Liu, and Hao Sun. "Physics-informed learning of governing equations from scarce data". In: *Nature communications* 12.1 (2021), pp. 1–13.

[9] Miles Cranmer. "Interpretable machine learning for science with PySR and SymbolicRegression. jl". In: *arXiv preprint arXiv:2305.01582* (2023).

[10] Roger Grimshaw. "Korteweg de-Vries Equation". In: *Nonlinear Waves in Fluids: Recent Advances and Modern Applications*. Ed. by Roger Grimshaw. Vienna: Springer Vienna, 2005, pp. 1–28. ISBN: 978-3-211-38025-3. DOI: 10.1007/3-211-38025-6_1. URL: https://doi.org/10.1007/3-211-38025-6_1.

[11] Daniel A. Messenger and David M. Bortz. "Weak SINDy for partial differential equations". In: *Journal of Computational Physics* 443 (2021), p. 110525. ISSN: 0021-9991. DOI: https://doi.org/10.1016/j.jcp.2021.110525. URL: https://www.sciencedirect.com/science/article/pii/S0021999121004204.

[12] Tommy Ohlsson. "The Klein–Gordon equation". In: *Relativistic Quantum Physics: From Advanced Quantum Mechanics to Introductory Quantum Field Theory*. Cambridge University Press, 2011, pp. 22–39.

[13] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". In: *Journal of Computational physics* 378 (2019), pp. 686–707.

[14] Samuel Rudy et al. "Data-Driven Identification of Parametric Partial Differential Equations". In: *SIAM Journal on Applied Dynamical Systems* 18.2 (2019), pp. 643–660. DOI: 10.1137/18M1191944. eprint: https://doi.org/10.1137/18M1191944. URL: https://doi.org/10.1137/18M1191944.

[15] Samuel H Rudy et al. "Data-driven discovery of partial differential equations". In: *Science advances* 3.4 (2017), e1602614.

[16] Robert Stephany and Christopher Earls. "PDE-LEARN: Using deep learning to discover partial differential equations from noisy, limited data". In: *Neural Networks* 174 (2024), p. 106242. ISSN: 0893-6080. DOI: https://doi.org/10.1016/j.neunet.2024.106242. URL: https://www.sciencedirect.com/science/article/pii/S0893608024001667.

[17] Robert Stephany and Christopher Earls. "PDE-READ: Human-readable partial differential equation discovery using deep learning". In: *Neural Networks* 154 (2022), pp. 360–382. ISSN: 0893-6080. DOI: https://doi.org/10.1016/j.neunet.2022.07.008. URL: https://www.sciencedirect.com/science/article/pii/S0893608022002660.